

应用依赖安全与合规分析平台PRD（实习生版）

文档版本：V1.0

核心目标：基于OWASP Dependency-Check构建集团统一的轻量级依赖安全与合规分析工具，输出完整的标准化文档体系，解决内部依赖管理混乱、安全与合规风险突出的问题

一、产品概述

1.1 项目背景

集团内部已累计数万款Java、前端及多语言应用，第三方组件依赖数量呈指数级增长。近一年因第三方依赖漏洞导致的安全事件占比超60%，同时许可证不合规问题可能引发知识产权纠纷和重大商业损失。

目前集团内部缺乏统一的依赖分析工具和标准化使用文档，各团队使用的零散工具存在功能不全、配置复杂、缺乏维护等问题。本项目以业界成熟的开源安全引擎为基础，通过深度复现、文档完善和轻量适配，打造一套可直接部署使用的内部解决方案。

1.2 产品定位

面向集团内部所有开发团队的轻量级、标准化依赖安全与合规分析工具，提供开箱即用的依赖漏洞检测、许可证合规分析能力，同时配套完整的使用和开发文档，大幅降低工具使用门槛。

1.3 核心价值

- 业务价值**：统一集团依赖分析标准，提前发现并修复依赖安全漏洞和许可证合规风险，保障应用稳定性和业务合规性
- 文档价值**：输出完整文档集，解决集团内部缺乏标准化依赖分析工具文档的痛点
- 技术价值**：基于业界成熟开源引擎构建，代码质量高、可扩展性强，为后续功能扩展奠定基础

二、核心参考与技术基础

2.1 基准依赖分析核心引擎

本项目所有核心分析能力均基于以下开源引擎构建，不修改其核心源码，仅做上层封装和适配：

资源名称	版本	核心能力	代码仓库
OWASP Dependency- Check	9.0.0	支持 10+语 言依赖 解析， 集成 NVD、 OSV等 多个漏 洞数据 源，提 供漏洞 检测和 许可证 分析	https://github.com/jeremylong/DependencyCheck

2.2 标准测试验证资源

所有功能必须通过以下官方开源测试工程的验证，确保功能正确性和适配性：

- Spring Boot Pet Clinic**：用于基础功能验证和核心流程测试
 - 代码仓库：<https://github.com/spring-projects/spring-petclinic>
- Spring Cloud Alibaba Nacos Example**：用于阿里技术栈适配测试和多模块项目扫描验证
 - 代码仓库：<https://github.com/alibaba/spring-cloud-alibaba/tree/2023.0.1.2/spring-cloud-alibaba-examples/nacos-example>
- Vulnerable Spring Boot App**：用于漏洞检测功能准确性验证和误报率测试
 - 代码仓库：<https://github.com/kiview/damn-vulnerable-spring-boot-app>

2.3 技术栈选型约束

所有开发必须严格遵循以下技术栈要求，不得随意引入额外组件：

后端核心

技术组件	版本	用途	约束说明
Spring Boot	3.2.x	后端基础 框架	采用单体架构， 禁止微服务拆分

MyBatis-Plus	3.5.5	数据访问层	全程使用H2嵌入式数据库，无需额外部署数据库
H2 Database	最新稳定版	数据存储	嵌入式运行，数据持久化到本地文件
Nacos	最新稳定版	配置中心	仅用于管理引擎配置参数，不做服务发现
阿里云OSS	-	对象存储	仅用于存储生成的扫描报告
OWASP Dependency-Check Core	9.0.0	核心分析引擎	直接引入官方核心库
Apache POI	最新稳定版	Excel报告生成	仅用于扩展报告格式

前端

- 基础框架：Vue 3.4.x
- UI组件库：Element Plus（仅使用基础组件）
- 构建工具：Vite

部署与运维

- 容器化：Docker、Docker Compose
- 服务器：阿里云ECS
- 反向代理：Nginx

三、产品功能需求

3.1 基准引擎核心能力

产品必须完整复现OWASP Dependency-Check 9.0.0的所有核心能力，并完成针对集团环境的适配优化：

- 1. 多语言依赖解析能力
 - 支持Java（Maven/Gradle）、JavaScript（NPM）、Python等主流语言的依赖解析
 - 支持单模块和多模块项目的依赖树构建
 - 支持排除指定路径或依赖的扫描
- 2. 漏洞检测能力
 - 集成NVD、OSV等官方漏洞数据源
 - 支持漏洞数据库的全量同步和增量更新
 - 支持漏洞等级划分（高危、中危、低危）和详情展示
 - 支持漏洞白名单配置，排除误报或无需修复的漏洞

3. 许可证合规分析能力

- 自动识别依赖组件的开源许可证类型
- 检测许可证冲突和不合规使用情况
- 生成许可证合规报告

4. 集团环境适配优化

- 针对集团网络环境优化漏洞数据库同步配置
- 针对集团主流技术栈（尤其是阿里技术栈）优化扫描参数
- 制定集团标准扫描配置模板，提升扫描速度和准确率

3.2 轻量Web服务能力

基于Spring Boot构建轻量级Web服务，提供可视化的依赖分析能力：

1. 项目管理能力

- 支持上传ZIP格式的项目压缩包
- 支持自定义扫描参数（排除路径、报告格式等）

2. 扫描任务管理能力

- 支持扫描任务的提交、取消和状态查询
- 支持扫描任务异步执行，避免阻塞主线程
- 支持扫描进度的实时展示

3. 报告管理能力

- 支持生成HTML、PDF、Excel三种格式的扫描报告
- 报告内容包含集团定制化信息
- 支持报告的在线预览和下载
- 报告存储在阿里云OSS，支持长期保存

4. 数据管理能力

- 支持扫描历史记录查询
- 支持扫描结果统计（漏洞数量、许可证问题数量等）
- 支持扫描结果导出
- 支持漏洞详情查询

3.3 前端交互能力

开发极简前端页面，实现完整的用户交互流程：

1. 用户认证：支持简单的用户登录（固定账号密码，无需注册功能）

2. 核心页面

- 项目上传与扫描任务提交页面
- 扫描任务列表页面（显示任务状态和操作按钮）
- 扫描结果详情页面（展示漏洞列表和许可证问题）
- 报告下载页面

3. 交互要求：页面简洁易用，交互流畅，无明显卡顿和Bug

3.4 部署与运维能力

1. 提供完整的Docker部署方案，支持一键部署
2. 支持前后端分离部署，使用Nginx作为反向代理
3. 系统部署后可通过公网访问，稳定运行

四、非功能需求

4.1 性能要求

1. 小型项目（代码量<1万行）扫描时间≤30秒
2. 中型项目（代码量1-10万行）扫描时间≤5分钟
3. 大型项目（代码量>10万行）扫描时间≤30分钟
4. 系统支持同时处理至少5个并发扫描任务
5. 扫描过程中内存占用≤4GB

4.2 兼容性要求

1. 支持JDK 17及以上版本运行
2. 支持主流浏览器（Chrome、Firefox、Edge）访问
3. 兼容集团主流技术栈（Spring Boot、Spring Cloud Alibaba等）

4.3 易用性要求

1. 提供完整的用户使用手册，包含详细的操作步骤和截图
2. 系统界面简洁直观，用户无需培训即可上手使用
3. 提供常见问题排查手册，帮助用户快速解决使用过程中遇到的问题

4.4 安全性要求

1. 系统仅用于集团内部使用，禁止对外开放
2. 禁止上传包含敏感信息的项目代码
3. 扫描报告仅对提交任务的用户可见
4. 严格遵守OWASP Dependency-Check的Apache 2.0开源协议

4.5 可维护性要求

2. 核心方法和类必须添加详细注释
3. 提供完整的开发手册，便于后续维护和功能扩展
4. 配置参数集中管理，支持动态更新，无需重启服务

五、文档需求

本项目文档价值突出，必须产出以下完整的文档体系：

1. 基础文档

- 《OWASP Dependency-Check 本地环境搭建手册》：包含环境准备、源码下载、编译构建、基础使用步骤及常见问题解决方案
- 《OWASP Dependency-Check 核心模块原理分析手册》：包含核心模块功能说明、代码结构、核心算法、调用流程及类图

2. 使用文档

- 《OWASP Dependency-Check 高级使用手册》：包含所有高级功能的使用方法和最佳实践
- 《集团依赖分析工具标准配置手册》：包含集团标准扫描配置模板及参数说明
- 《OWASP Dependency-Check 常见问题排查手册》：包含所有常见问题的排查步骤和解决方案
- 《集团依赖安全与合规管理最佳实践》：包含依赖安全管理、许可证合规管理及CI/CD集成的最佳实践
- 《系统用户使用手册》：包含Web系统的详细操作步骤和截图

3. 开发文档

- 《轻量依赖分析Web服务开发手册》：包含框架搭建、引擎集成、接口开发、功能实现的详细步骤
- 《前端开发手册》：包含前端项目搭建、页面开发、前后端联调的详细步骤
- 《依赖分析报告格式说明》：包含报告生成、存储、下载的实现说明及报告示例

4. 项目总览文档

- 项目总览文档：介绍项目背景、目标、架构和功能

六、交付物标准

6.1 代码交付标准

2. 代码符合开发规范，核心逻辑有详细注释
3. 所有功能必须经过测试，无严重Bug和功能缺陷
4. 提供完整的部署脚本（Dockerfile、docker-compose.yml）

6.2 文档交付标准

1. 所有文档采用Markdown格式编写，最终导出为PDF
2. 文档必须包含详细的操作步骤和清晰的截图，截图需标注关键操作区域
3. 代码示例必须完整可运行，包含必要的注释

七、风险与注意事项

- 1. **网络风险**：NVD漏洞数据库同步可能因网络问题失败，需提前准备离线同步方案或使用国内镜像
- 2. **性能风险**：扫描大型项目可能出现内存溢出，需提前调整JVM参数，优化扫描配置
- 3. **适配风险**：部分阿里技术栈组件可能无法被准确识别，需记录问题并给出临时解决方案
- 4. **开源协议风险**：严格遵守OWASP Dependency-Check的Apache 2.0开源协议，不得用于商业用途

八、参考资料汇总

资料类型	名称	链接/地址
基准引擎	OWASP Dependency-Check 9.0.0	https://github.com/jeremylong/DependencyCheck
官方文档	OWASP Dependency-Check 官方文档	https://jeremylong.github.io/DependencyCheck/
测试工程1	Spring Boot Pet Clinic	https://github.com/spring-projects/spring-petclinic
测试工程2	Spring Cloud Alibaba Nacos Example	https://github.com/alibaba/spring-cloud-alibaba/tree/2023.0.1.2/spring-cloud-alibaba-examples/nacos-example
测试工程3	Vulnerable Spring Boot App	https://github.com/kiview/damn-vulnerable-spring-boot-app